LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 10
“PENCARIAN DATA TERSTRUKTUR”

DISUSUN OLEH:

ITRHOH ANGGUN PAMUNGKAS

109082500117

S1 IF-13-02

DOSEN:

Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2024/2025

DASAR TEORI

Pencarian (searching) adalah proses untuk menemukan suatu data di dalam kumpulan data. Biasanya data disimpan dalam bentuk array, yaitu sekumpulan nilai dengan tipe yang sama yang disusun secara berurutan dan bisa diakses menggunakan indeks.

Salah satu jenis pencarian yang paling sederhana adalah mencari nilai maksimum (terbesar) dan minimum (terkecil). Cara kerjanya sangat basic: ambil data pertama sebagai nilai awal, lalu bandingkan dengan data berikutnya satu per satu. Kalau ketemu yang lebih besar atau lebih kecil, nilai tersebut diganti. Proses ini dilakukan sampai semua data dicek.

Metode ini termasuk dalam pencarian sekuensial (linear), karena dilakukan secara berurutan dari awal sampai akhir data. Metode ini sederhana dan mudah dipahami, walaupun tidak selalu paling cepat.

Selain itu, dalam pengolahan data sering juga dihitung nilai rata-rata (mean). Rata-rata didapat dengan menjumlahkan semua data, lalu dibagi dengan jumlah data. Perhitungan ini juga menggunakan perulangan seperti pada pencarian nilai maksimum dan minimum.

SOAL LATIHAN

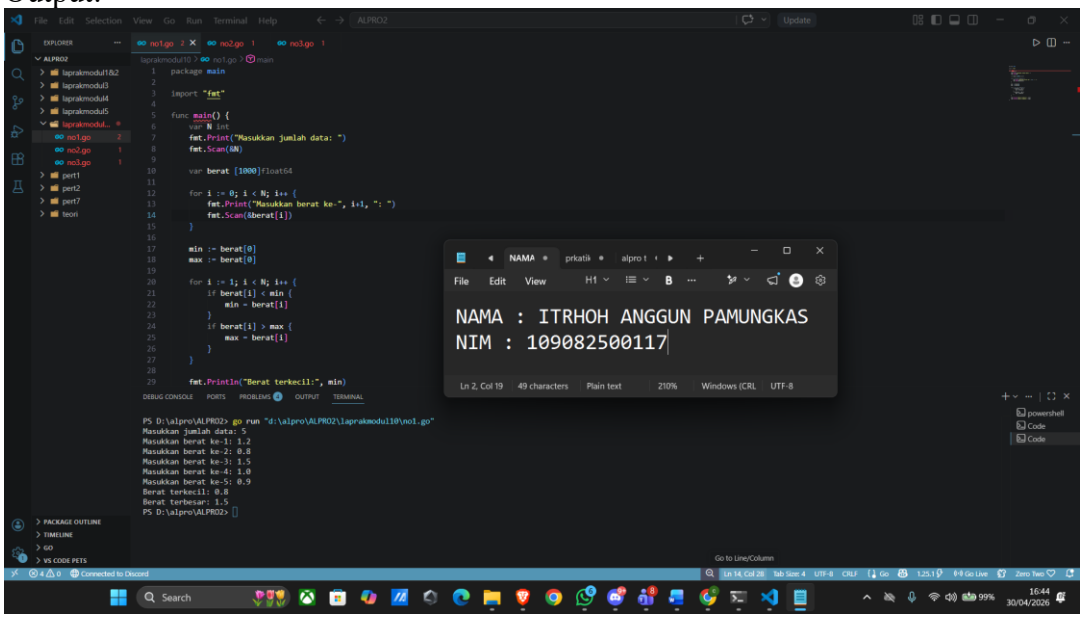
Statement Pencarian Data Terstruktur

1.

Source Code:

```
package main

import "fmt"

func main() {
    var N int
    fmt.Print("Masukkan jumlah data: ")
    fmt.Scan(&N)

    var berat [1000]float64

    for i := 0; i < N; i++ {
        fmt.Print("Masukkan berat ke-", i+1, ": ")
        fmt.Scan(&berat[i])
    }

    min := berat[0]
    max := berat[0]

    for i := 1; i < N; i++ {
        if berat[i] < min {
            min = berat[i]
        }
        if berat[i] > max {
            max = berat[i]
        }
    }

    fmt.Println("Berat terkecil:", min)
    fmt.Println("Berat terbesar:", max)
}
```

Output:



Deskripsi Program:

Program dimulai dengan meminta jumlah data (N), lalu menyimpan berat kelinci ke dalam array. Setelah itu, nilai pertama dijadikan sebagai nilai minimum dan maksimum awal. Program kemudian melakukan perulangan untuk membandingkan setiap data berikutnya. Jika ditemukan nilai yang lebih kecil dari minimum, maka nilai minimum diperbarui, dan jika ditemukan nilai yang lebih besar dari maksimum, maka nilai maksimum diperbarui. Setelah semua data dicek, program menampilkan nilai terkecil dan terbesar.

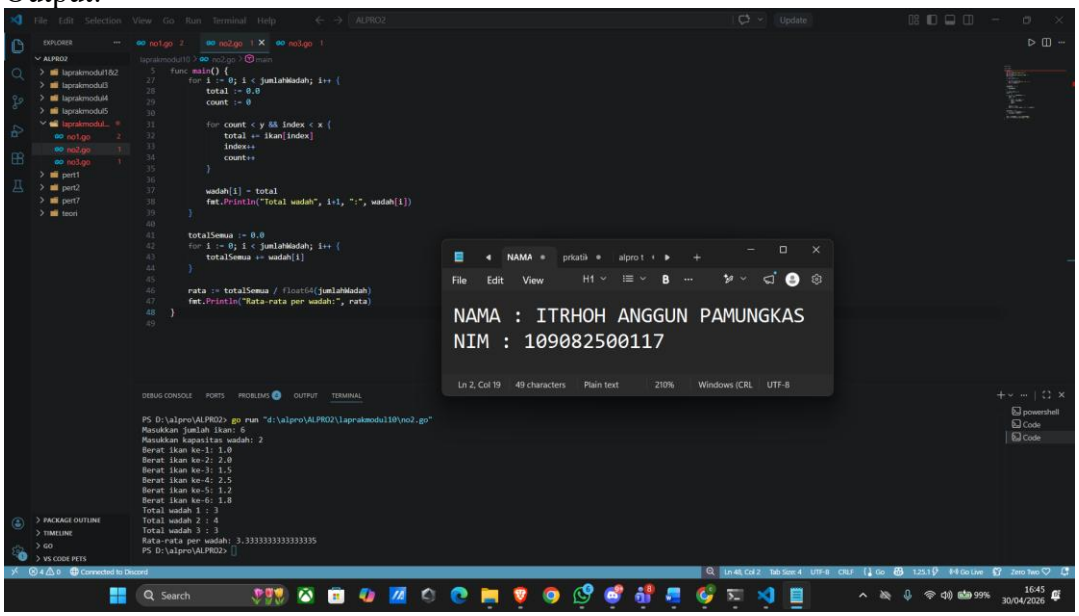
Statement Pencarian Data Terstruktur

2.

Source Code:

```
package main

import "fmt"

func main() {
    var x, y int
    fmt.Print("Masukkan jumlah ikan: ")
    fmt.Scan(&x)
    fmt.Print("Masukkan kapasitas wadah: ")
    fmt.Scan(&y)

    var ikan [1000]float64

    for i := 0; i < x; i++ {
        fmt.Print("Berat ikan ke-", i+1, ": ")
        fmt.Scan(&ikan[i])
    }

    jumlahWadah := x / y
    if x%y != 0 {
        jumlahWadah++
    }

    var wadah [1000]float64
    index := 0

    for i := 0; i < jumlahWadah; i++ {
        total := 0.0
        count := 0

        for count < y && index < x {
            total += ikan[index]
            index++
            count++
        }

        wadah[i] = total
        fmt.Println("Total wadah", i+1, ":", wadah[i])
    }

    totalSemua := 0.0
    for i := 0; i < jumlahWadah; i++ {
        totalSemua += wadah[i]
    }

    rata := totalSemua / float64(jumlahWadah)
    fmt.Println("Rata-rata per wadah:", rata)
}
```

Output:



Deskripsi Program:

Program meminta jumlah ikan (x) dan kapasitas wadah (y), lalu menyimpan data berat ikan ke dalam array. Program menghitung jumlah wadah yang dibutuhkan berdasarkan banyaknya ikan dan kapasitas tiap wadah. Selanjutnya, program mengelompokkan ikan ke dalam wadah secara berurutan dan menjumlahkan berat ikan di setiap wadah. Setelah semua wadah terisi, program menghitung total seluruh berat wadah dan membaginya dengan jumlah wadah untuk mendapatkan rata-rata, lalu menampilkan hasilnya.

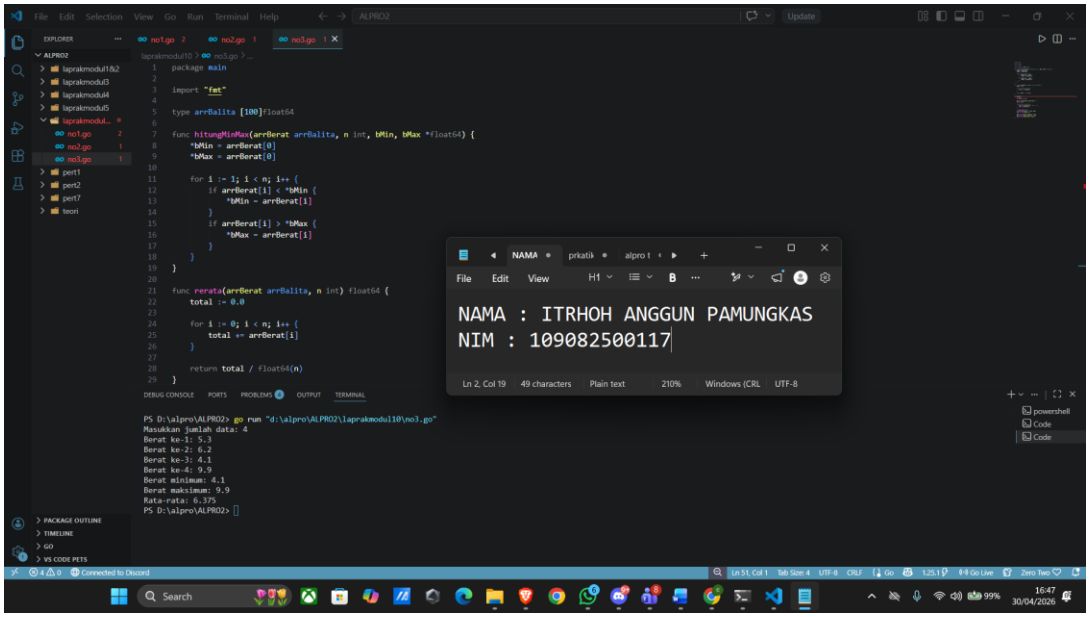
Statement Pencarian Data Terstruktur

3.

Source Code:

```
package main
import "fmt"

type arrBalita [100]float64

func hitungMinMax(arrBerat arrBalita, n int, bMin, bMax
*float64) {
    *bMin = arrBerat[0]
    *bMax = arrBerat[0]

    for i := 1; i < n; i++ {
        if arrBerat[i] < *bMin {
            *bMin = arrBerat[i]
        }
        if arrBerat[i] > *bMax {
            *bMax = arrBerat[i]
        }
    }
}

func rerata(arrBerat arrBalita, n int) float64 {
    total := 0.0

    for i := 0; i < n; i++ {
        total += arrBerat[i]
    }
    return total / float64(n)
}

func main() {
    var n int
    var berat arrBalita

    fmt.Print("Masukkan jumlah data: ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Print("Berat ke-", i+1, ": ")
        fmt.Scan(&berat[i])
    }

    var min, max float64
    hitungMinMax(berat, n, &min, &max)
    rata := rerata(berat, n)

    fmt.Println("Berat minimum:", min)
    fmt.Println("Berat maksimum:", max)
    fmt.Println("Rata-rata:", rata)
}
```

Output:



Deskripsi Program:

Program meminta jumlah data berat balita dan menyimpannya dalam array. Fungsi hitungMinMax digunakan untuk mencari nilai minimum dan maksimum dengan cara membandingkan setiap data dalam array. Fungsi rerata digunakan untuk menghitung rata-rata dengan menjumlahkan semua data lalu dibagi jumlah data. Setelah kedua fungsi dipanggil, program menampilkan hasil minimum, maksimum, dan rata-rata berat balita.

DAFTAR PUSTAKA

https://informatika.stei.itb.ac.id/~rinaldi.munir/Buku/Algoritma%20dan%20Pemrograman%20II%20%28Edisi%203%29/Bab1-Pencarian.pdf

https://repository.unikom.ac.id/36305/1/08%20Array.pdf

https://www.slideshare.net/slideshow/pertemuan-11-tehnik-searching/87582710